Agentic AI Engineering Internship 2026

Week 5 Comprehensive Report

AI Fundamentals & Agentic AI Foundations: Multi-Agent Systems

Prepared By:
Ghafoor

Date:
August 9, 2026

CalderR.

RETHINKING · HOW · WORK · WORKS

Contents

1 Executive Summary

This comprehensive report details the theoretical concepts, practical lab exercises, and ambitious multi-agent projects completed during Week 5 of the CalderR Agentic AI Engineering Internship. Transitioning from single-agent systems to coordinated teams, the focus of the week was orchestrating teams of AI agents to solve complex problems collaboratively.

The week emphasized advanced architectures including orchestrator-worker, supervisor patterns, and hierarchical teams. By engineering structured agent-to-agent communication via typed message passing and implementing robust conflict resolution and failure handling mechanisms, the foundation for highly resilient production-ready multi-agent systems was established.

The culmination of these learnings is showcased in two sophisticated projects: the **Autonomous Competitive Intelligence Agent** (Intermediate) and the **Autonomous AI Research Lab** (Production). These solutions demonstrate true parallel agent execution, confidence-weighted consensus, dynamic agent assembly, and explicit observability.

2 Conceptual Mastery & Assessment

The curriculum this week demanded a deep dive into the architectural principles and operational patterns that govern effective multi-agent collaboration.

2.1 Multi-Agent Architectures & Communication

- **Architectural Patterns:** Mastered orchestrator-worker models, supervisor chains, peer networks, and hierarchical teams. Identified when a single agent outperforms a team versus when task delegation is crucial.
- **Typed Message Passing:** Replaced raw strings with structured Pydantic schemas (TaskRequest, TaskResult, ErrorReport, Handoff) for inter-agent communication, drastically improving reliability and type safety.
- **Specialization Design:** Engineered specialized agent personas with distinct tooling, authority scopes, and domain expertise.
- **Failure Handling & Consensus:** Implemented resilient supervisor logic that gracefully handles timeouts and low-confidence responses through retry or fallback routing. Engineered debate patterns and confidence-weighted consensus to adjudicate contradictions explicitly.
- **Observability:** Instrumenting systems with per-agent logging for token usage, latency, and decision tracking via LangSmith and structured audit trails.

3 Production Project: Autonomous AI Research Lab

Architectural Overview

A fully autonomous research system that takes a natural-language research question, dynamically assembles a team of specialist agents based on the domain, executes a multi-phase research process (hypothesis, evidence gathering, critic, synthesis, peer review), and publishes a professional report without human intervention.

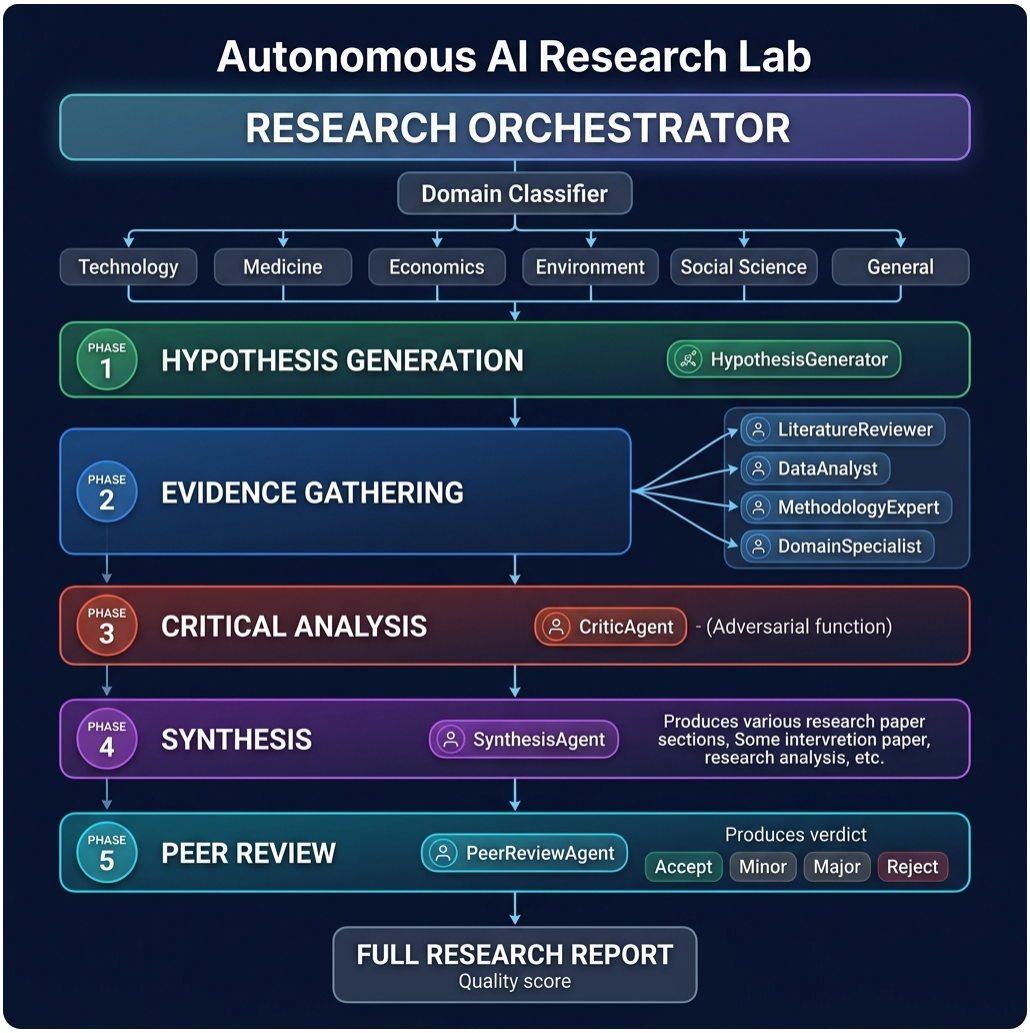

Figure 1: Production Project: Autonomous AI Research Lab Architecture

3.1 Technical Implementation

- **Dynamic Agent Assembly:** The Domain Classifier dynamically selects and instantiates 3–5 specialized Evidence Agents tailored to the research topic.
- **Multi-Phase Orchestration:** Utilized LangGraph subgraphs to manage a hierarchical flow: hypothesis generation → parallel evidence gathering → critical review

→ synthesis → peer review.

- **Critic & Peer Review Patterns:** Implemented a Critic Agent to explicitly challenge weak links in evidence, and a Peer Review Agent to act as a second-pass quality check before publication.
- **Comprehensive Observability:** Features a FastAPI REST API, a Streamlit UI demonstrating real-time phase progress, and comprehensive LangSmith tracing for auditing decisions.

3.2 System Demonstrations

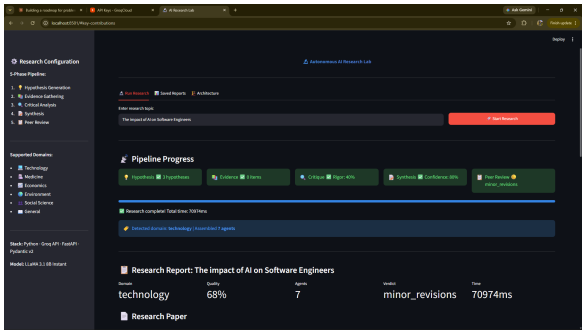

Figure 2: Research Domain Classification

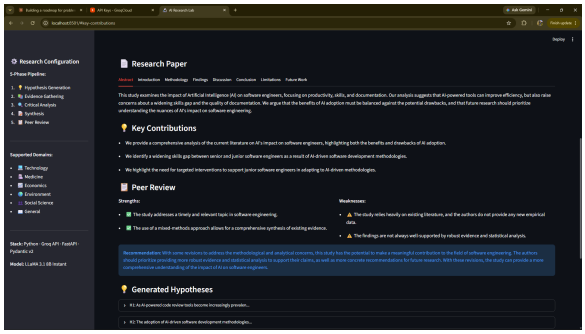

Figure 3: Dynamic Agent Assembly

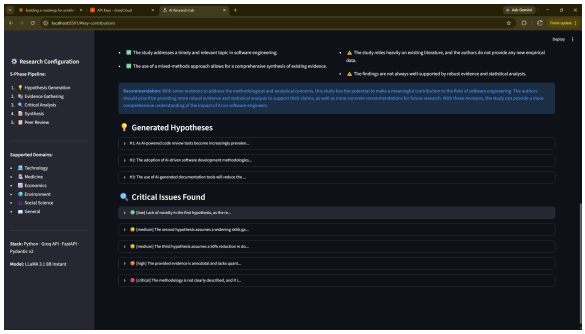

Figure 4: RAG Evidence Gathering

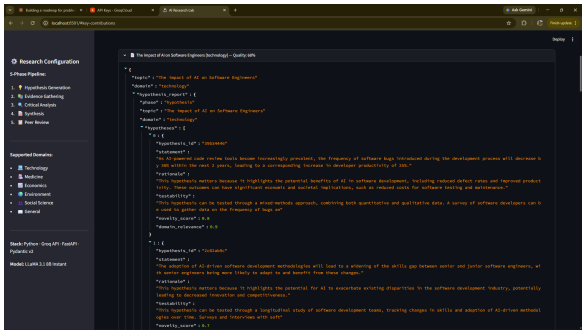

Figure 5: Final Published Research Report

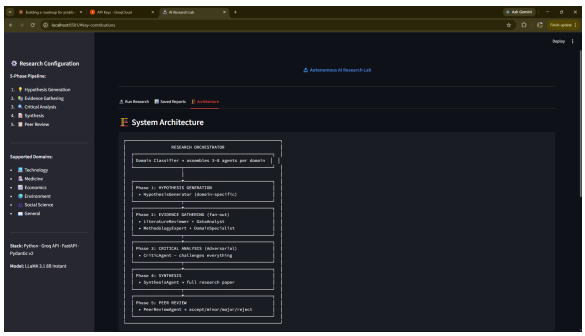

Figure 6: System Architecture Diagram

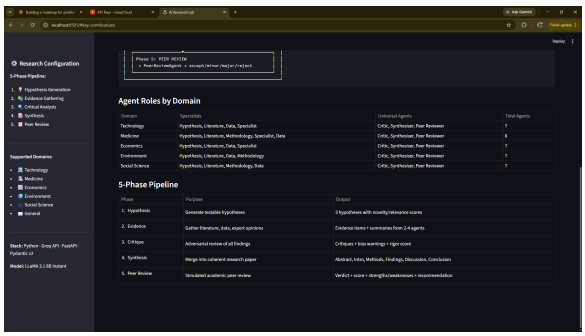

Figure 7: Agent Roles & Pipeline

4 Intermediate Project: Autonomous Competitive Intelligence Agent

Architectural Overview

A multi-agent system designed to autonomously research a company from multiple angles (market, product, tech stack, news, sentiment). It utilizes parallel fan-out execution, incorporates conflict resolution logic, and aggregates findings into a structured, professional intelligence briefing.

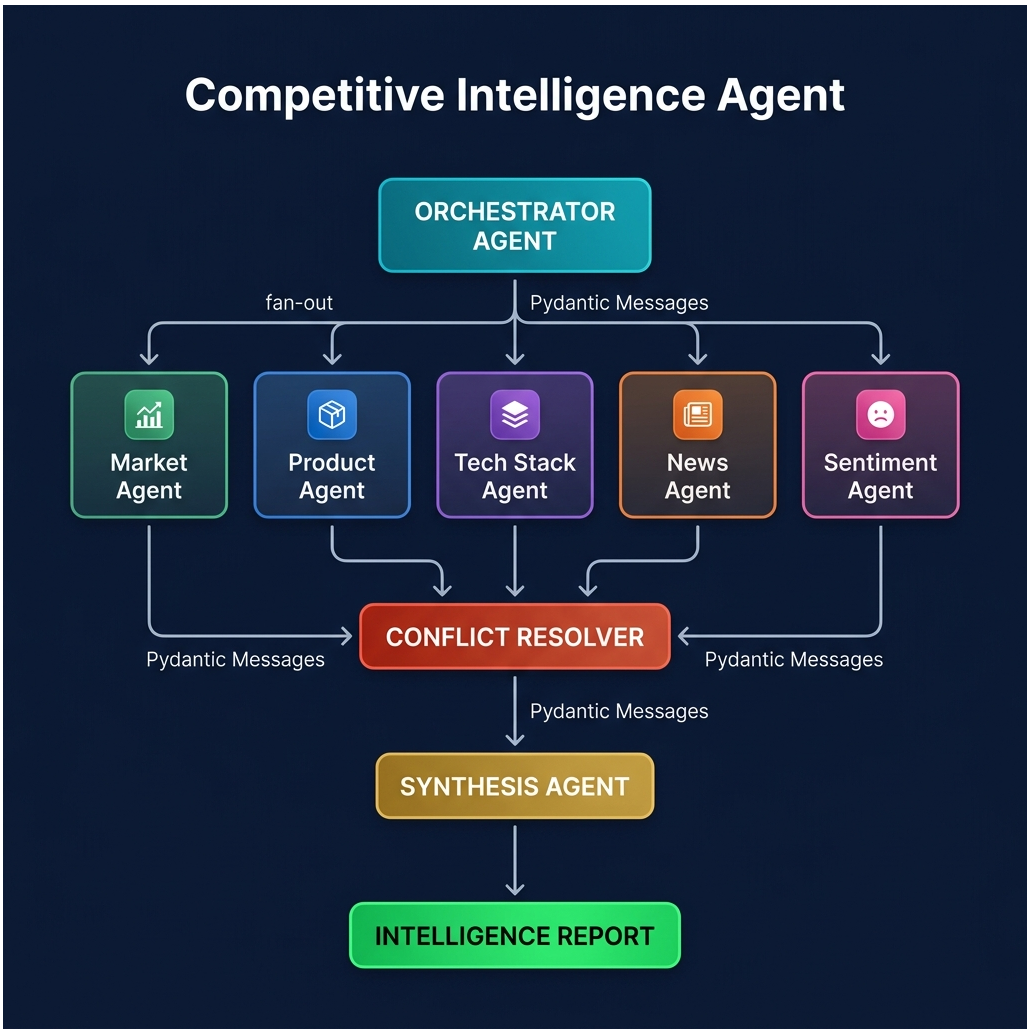

Figure 8: Intermediate Project: Competitive Intelligence System Architecture

4.1 Technical Implementation

- **Parallel Fan-Out Execution:** The Orchestrator Agent decomposes the research strategy and concurrently dispatches tasks to five specialized agents (Market, Product, Tech Stack, News, Sentiment).
- **Conflict Adjudication:** Embedded a Conflict Resolver agent that detects and adjudicates contradictions between individual agent findings before final synthesis.

- **Structured Typed Outputs:** Ensured every agent strictly adheres to Pydantic schemas, enabling reliable aggregation and generation of a clean, non-redundant Markdown report exposed via FastAPI and Streamlit.

4.2 System Demonstrations

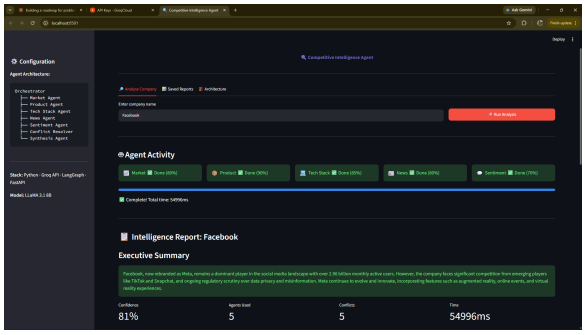

Figure 9: Parallel Dispatch & Agent Activity

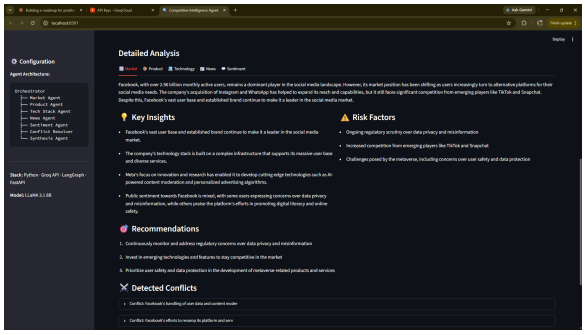

Figure 10: Conflict Resolution Log & Detailed Analysis

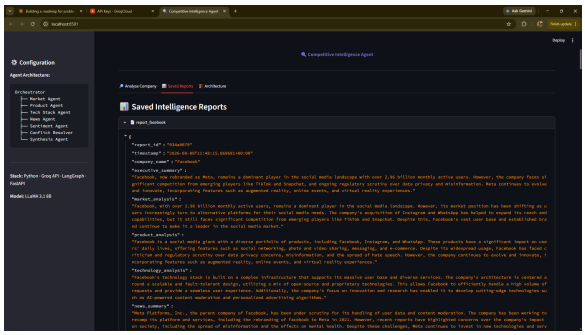

Figure 11: Structured Findings Aggregation

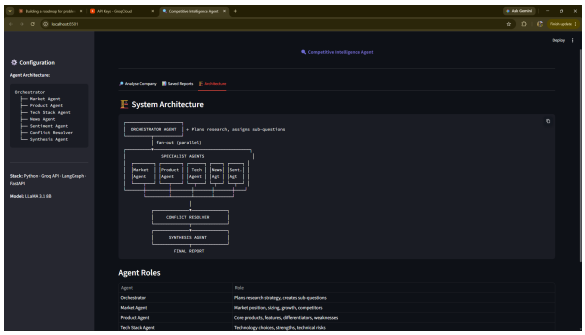

Figure 12: System Architecture Representation

5 Foundational Labs Deep-Dive

5.1 Lab 5.1: Typed Message Bus

Developed the backbone for inter-agent communication by enforcing strict Pydantic message schemas over raw strings.

- **Schema Validation:** Designed TaskRequest, TaskResult, ErrorReport, and Handoff models, ensuring malformed messages are caught by validation errors.
- **Reliable Handoffs:** Implemented three agents communicating exclusively over this in-memory typed message bus, guaranteeing type-safety.

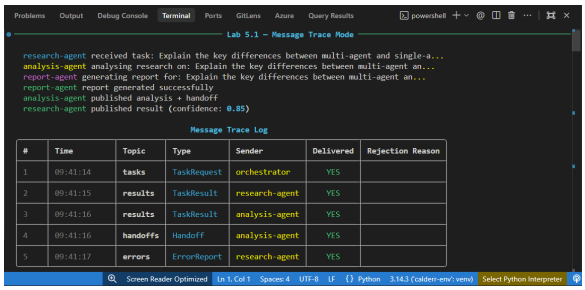

Figure 13: Typed Schema Validation

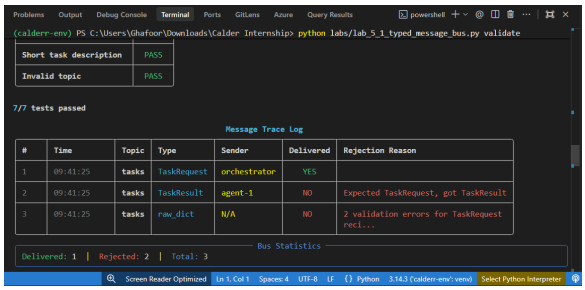

Figure 14: Agent Communication via Bus

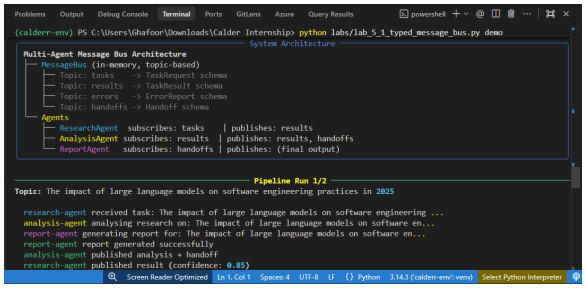

Figure 15: Handoff Execution

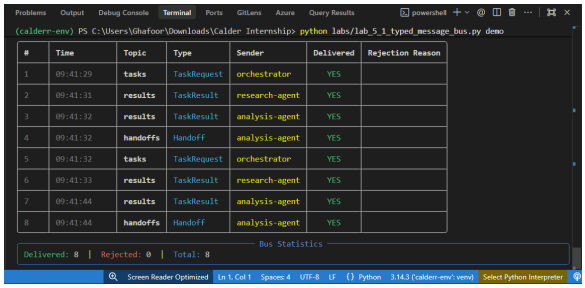

Figure 16: Error Handling in Messaging

5.2 Lab 5.2: Supervisor with Failure Recovery

Built resilience into multi-agent systems by implementing a Supervisor Agent that gracefully manages routine failures from its delegate specialists.

- **Fault Injection:** Tested the system against simulated timeouts and low-confidence responses.
- **Graceful Degradation:** Designed the supervisor to log failure reasons, route to fallback agents, and synthesize partial results to prevent system crashes.

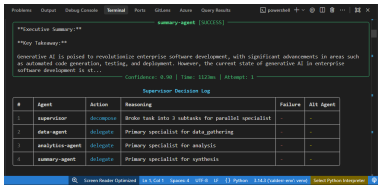

Figure 17: Delegation Trace

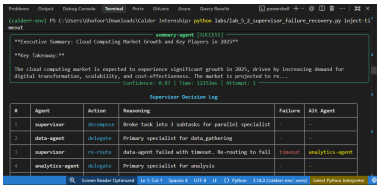

Figure 18: Timeout Detection

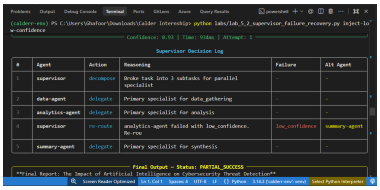

Figure 19: Fallback Routing Logic

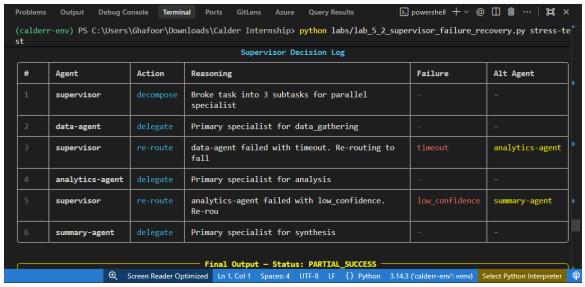

Figure 20: Supervisor Decision Log

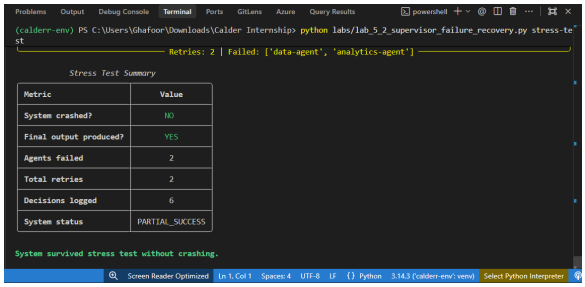

Figure 21: Gracefully Degraded Response

5.3 Lab 5.3: Consensus Engine

Engineered a debate and voting system to handle complex queries where multiple specialist agents form independent opinions.

- **Confidence-Weighted Voting:** The Consensus Agent aggregates answers by multiplying opinions with confidence scores (0–1).
- **Multi-Round Debate:** Configured thresholds (60%) that trigger a second debate round among the top 2 agents if consensus isn't reached, explicitly highlighting dissent in the final summary.

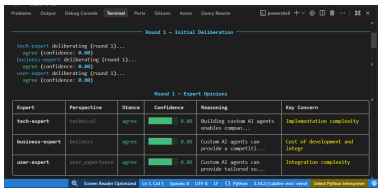

Figure 22: Initial Independent Opinions

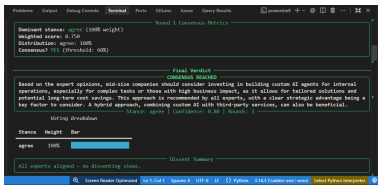

Figure 23: First Voting Round

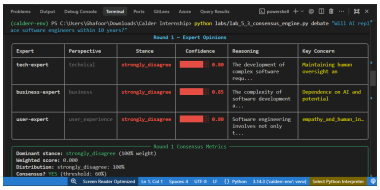

Figure 24: Triggering Round Two

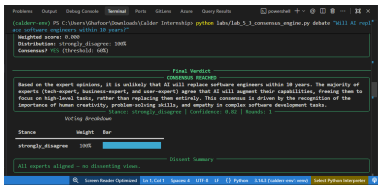

Figure 25: Second Round Debate

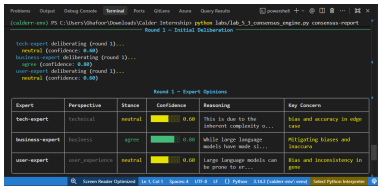

Figure 26: Consensus Aggregation

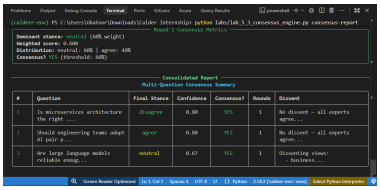

Figure 27: Final Output & Dissent

6 Conclusion & Next Steps

Week 5 represented a significant paradigm shift from engineering single agents to orchestrating complex, autonomous teams. By mastering graph-based delegation, enforcing typed communication, designing specialized personas, and implementing robust failure recovery and conflict adjudication, the projects developed this week mirror genuine production architectures. These advanced multi-agent patterns form the core capabilities required to solve sophisticated real-world engineering challenges autonomously.